> Who knows git?

I lost 4 hours of work after accidental `git reset`

I lost 4 hours of work after accidental `git reset`

Or my AI helped me with it

The Git Panic Moment

- "I lost my commits after a reset!"
- "Which commit broke the build?"
- "Who wrote this code???"
- "My PR history is a mess!"

All of them (and mode) are solvable

The Promise: Git Zen

What if:

- AI could recover lost commits for you?
- AI could find the bug-introducing commit (semi)automatically?
- AI could clean up your messy history?

From Git Panic to Git Zen

With a Little Help from AI

Pasha Finkelshteyn
@asm0di0
https://asm0dey.site

Three panics, three tools

Reflog → Bisect → Interactive rebase

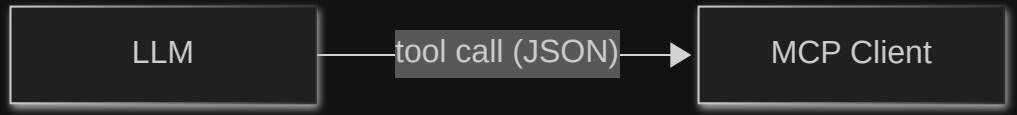
From the shortest tool we wrote to the gnarliest one.

MCP in 30 seconds

AI models can't *just* run your code.

They need a protocol to discover tools, call them, and get structured results back.

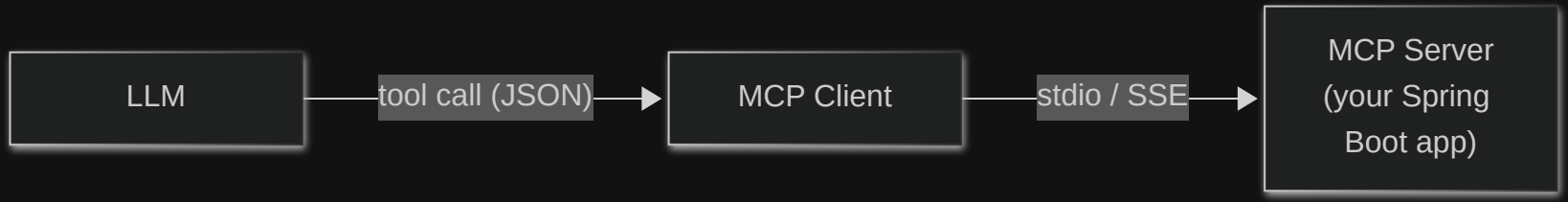
Model Context Protocol (MCP) is that protocol.



MCP in 30 seconds

AI models can’t just run your code. They need a protocol to discover tools, call them, and get structured results back.

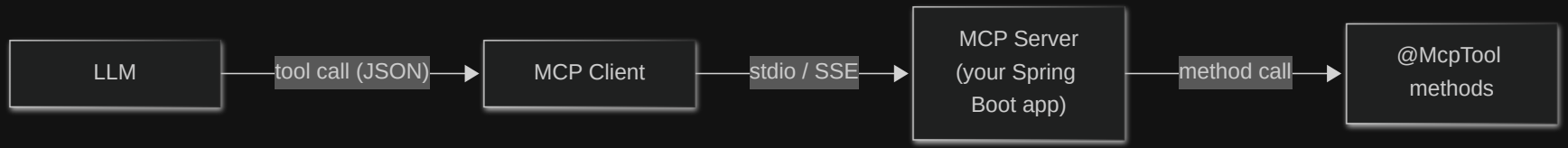
Model Context Protocol (MCP) is that protocol.



MCP in 30 seconds

AI models can’t just run your code. They need a protocol to discover tools, call them, and get structured results back.

Model Context Protocol (MCP) is that protocol.



MCP in 30 seconds

AI models can’t just run your code. They need a protocol to discover tools, call them, and get structured results back.

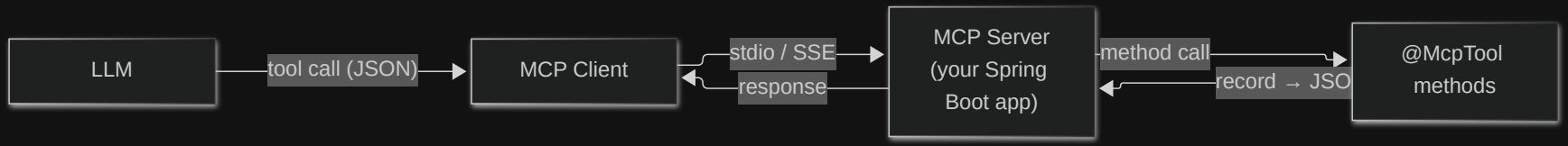
Model Context Protocol (MCP) is that protocol.



MCP in 30 seconds

AI models can’t just run your code. They need a protocol to discover tools, call them, and get structured results back.

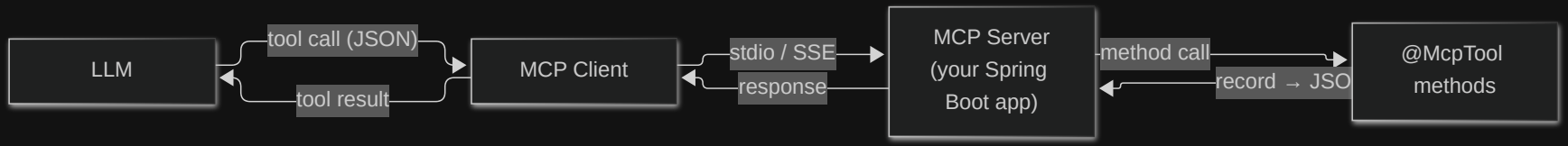
Model Context Protocol (MCP) is that protocol.



MCP in 30 seconds

AI models can’t just run your code. They need a protocol to discover tools, call them, and get structured results back.

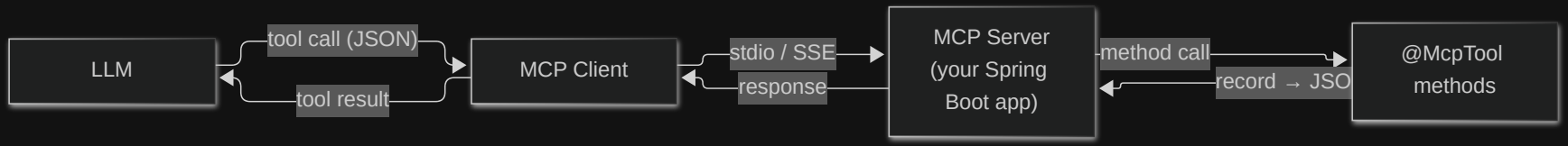
Model Context Protocol (MCP) is that protocol.



MCP in 30 seconds

AI models can’t just run your code. They need a protocol to discover tools, call them, and get structured results back.

Model Context Protocol (MCP) is that protocol.



We write the methods on the right. Spring AI + MCP handle everything in the middle.

The project at a glance

Two dependencies:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server</artifactId>
</dependency>
<dependency>
  <groupId>org.eclipse.jgit</groupId>
  <artifactId>org.eclipse.jgit</artifactId>
</dependency>
```

The project at a glance

Two dependencies:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server</artifactId>
</dependency>
<dependency>
  <groupId>org.eclipse.jgit</groupId>
  <artifactId>org.eclipse.jgit</artifactId>
</dependency>
```

The project at a glance

Two dependencies:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server</artifactId>
</dependency>
<dependency>
  <groupId>org.eclipse.jgit</groupId>
  <artifactId>org.eclipse.jgit</artifactId>
</dependency>
```

stdio MCP server — not a web app:

```
spring:
  main:
    web-application-type: none
    banner-mode: off
  application:
    name: git-mcp
  ai:
    mcp:
      server:
        stdio: true
        name: git
        version: 1.0.0
        type: SYNC
        annotation-scanner:
          enabled: true
```

The project at a glance

Two dependencies:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server</artifactId>
</dependency>
<dependency>
  <groupId>org.eclipse.jgit</groupId>
  <artifactId>org.eclipse.jgit</artifactId>
</dependency>
```

stdio MCP server — not a web app:

```
spring:
  main:
    web-application-type: none
    banner-mode: off
  application:
    name: git-mcp
  ai:
    mcp:
      server:
        stdio: true
        name: git
        version: 1.0.0
        type: SYNC
        annotation-scanner:
          enabled: true
```

The project at a glance

Two dependencies:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server</artifactId>
</dependency>
<dependency>
  <groupId>org.eclipse.jgit</groupId>
  <artifactId>org.eclipse.jgit</artifactId>
</dependency>
```

stdio MCP server — not a web app:

```
spring:
  main:
    web-application-type: none
    banner-mode: off
  application:
    name: git-mcp
  ai:
    mcp:
      server:
        stdio: true
        name: git
        version: 1.0.0
        type: SYNC
        annotation-scanner:
          enabled: true
```

The project at a glance

Two dependencies:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server</artifactId>
</dependency>
<dependency>
  <groupId>org.eclipse.jgit</groupId>
  <artifactId>org.eclipse.jgit</artifactId>
</dependency>
```

stdio MCP server — not a web app:

```
spring:
  main:
    web-application-type: none
    banner-mode: off
  application:
    name: git-mcp
  ai:
    mcp:
      server:
        stdio: true
        name: git
        version: 1.0.0
        type: SYNC
        annotation-scanner:
          enabled: true
```

The project at a glance

Two dependencies:

```xml
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server</artifactId>
</dependency>
<dependency>
  <groupId>org.eclipse.jgit</groupId>
  <artifactId>org.eclipse.jgit</artifactId>
</dependency>
```

stdio MCP server — not a web app:

```yaml
spring:
  main:
    web-application-type: none
    banner-mode: off
  application:
    name: git-mcp
  ai:
    mcp:
      server:
        stdio: true
        name: git
        version: 1.0.0
        type: SYNC
        annotation-scanner:
          enabled: true
```

The project at a glance

Two dependencies:

```xml
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server</artifactId>
</dependency>
<dependency>
  <groupId>org.eclipse.jgit</groupId>
  <artifactId>org.eclipse.jgit</artifactId>
</dependency>
```

stdio MCP server — not a web app:

```yaml
spring:
  main:
    web-application-type: none
    banner-mode: off
  application:
    name: git-mcp
  ai:
    mcp:
      server:
        stdio: true
        name: git
        version: 1.0.0
        type: SYNC
        annotation-scanner:
          enabled: true
```

Panic #1: "I lost my commits after a reset"

You ran `git reset --hard HEAD~5` on the wrong branch. Your last afternoon of work is gone from the log.

Panic #1: "I lost my commits after a reset"

You ran `git reset --hard HEAD~5` on the wrong branch. Your last afternoon of work is gone from the log.

Except it isn’t.

Git keeps every HEAD movement in the reflog for 90 days.

The commits are still in the object database, they’re just not reachable from any ref.

Git's answer: `git reflog`

```
$ git log --oneline
abc1234 Some old commit        # ← your recent work is gone
# 😱 The last 5 commits vanished from the log!

$ git reflog
abc1234 HEAD@{0}: reset: moving to HEAD~5
def5678 HEAD@{1}: commit: Add user authentication

$ git reset --hard def5678 # or HEAD@{1}
# Your work is restored.
```

Git's answer: `git reflog`

```
$ git log --oneline
abc1234 Some old commit        # ← your recent work is gone
# 😱 The last 5 commits vanished from the log!

$ git reflog
abc1234 HEAD@{0}: reset: moving to HEAD~5
def5678 HEAD@{1}: commit: Add user authentication

$ git reset --hard def5678 # or HEAD@{1}
# Your work is restored.
```

Git's answer: `git reflog`

```
$ git log --oneline
abc1234 Some old commit        # ← your recent work is gone
# 😱 The last 5 commits vanished from the log!

$ git reflog
abc1234 HEAD@{0}: reset: moving to HEAD~5
def5678 HEAD@{1}: commit: Add user authentication

$ git reset --hard def5678 # or HEAD@{1}
# Your work is restored.
```

Git's answer: `git reflog`

```
$ git log --oneline
abc1234 Some old commit        # ← your recent work is gone
# 😱 The last 5 commits vanished from the log!

$ git reflog
abc1234 HEAD@{0}: reset: moving to HEAD~5
def5678 HEAD@{1}: commit: Add user authentication

$ git reset --hard def5678 # or HEAD@{1}
# Your work is restored.
```

Git's answer: `git reflog`

```
$ git log --oneline
abc1234 Some old commit        # ← your recent work is gone
# 😱 The last 5 commits vanished from the log!

$ git reflog
abc1234 HEAD@{0}: reset: moving to HEAD~5
def5678 HEAD@{1}: commit: Add user authentication

$ git reset --hard def5678 # or HEAD@{1}
# Your work is restored.
```

Two commands, two seconds

Git's answer: `git reflog`

```
$ git log --oneline
abc1234 Some old commit        # ← your recent work is gone
# 😱 The last 5 commits vanished from the log!

$ git reflog
abc1234 HEAD@{0}: reset: moving to HEAD~5
def5678 HEAD@{1}: commit: Add user authentication

$ git reset --hard def5678 # or HEAD@{1}
# Your work is restored.
```

Two commands, two seconds

How good are you under pressure?

Java + MCP: `GitReflogService`

Our first look at the pattern. `@McpTool` on a Spring bean method, `@McpToolParam` on every argument, and Spring AI does the rest of the wiring.

```
public record ReflogEntryRecord(
        int index,
        String shortHash,
        String fullHash,
        String author,
        String timestamp,
        String message
) {}

@McpTool(name = "git_reflog",
    description = "List, show, and reset references "
                + "to previous states.")
public ReflogOperationResult gitReflog(
        @McpToolParam("Action: list, show, reset.") String action,
        @McpToolParam("Absolute path to the Git repo") String directory,
        @McpToolParam(value = "Git reference (default: HEAD).",
                required = false) String reference,
        @McpToolParam(value = "Entry index (0-based; negatives "
```

Java + MCP: `GitReflogService`

Our first look at the pattern. `@McpTool` on a Spring bean method, `@McpToolParam` on every argument, and Spring AI does the rest of the wiring.

```
public record ReflogEntryRecord(
        int index,
        String shortHash,
        String fullHash,
        String author,
        String timestamp,
        String message
) {}

@McpTool(name = "git_reflog",
    description = "List, show, and reset references "
                + "to previous states.")
public ReflogOperationResult gitReflog(
        @McpToolParam("Action: list, show, reset.") String action,
        @McpToolParam("Absolute path to the Git repo") String directory,
        @McpToolParam(value = "Git reference (default: HEAD).",
                required = false) String reference,
        @McpToolParam(value = "Entry index (0-based; negatives "
```

Java + MCP: `GitReflogService`

Our first look at the pattern. `@McpTool` on a Spring bean method, `@McpToolParam` on every argument, and Spring AI does the rest of the wiring.

```
        String shortHash,
        String fullHash,
        String author,
        String timestamp,
        String message
) {}

@McpTool(name = "git_reflog",
    description = "List, show, and reset references "
                + "to previous states.")
public ReflogOperationResult gitReflog(
        @McpToolParam("Action: list, show, reset.") String action,
        @McpToolParam("Absolute path to the Git repo") String directory,
        @McpToolParam(value = "Git reference (default: HEAD).",
                required = false) String reference,
        @McpToolParam(value = "Entry index (0-based; negatives "
                + "count from end).", required = false) Integer index,
        @McpToolParam(value = "Max entries.",
                required = false) Integer limit) { ... }
```

Java + MCP: `GitReflogService`

Our first look at the pattern. `@McpTool` on a Spring bean method, `@McpToolParam` on every argument, and Spring AI does the rest of the wiring.

```
            String shortHash,
            String fullHash,
            String author,
            String timestamp,
            String message
    ) {}

    @McpTool(name = "git_reflog",
        description = "List, show, and reset references "
                    + "to previous states.")
    public ReflogOperationResult gitReflog(
            @McpToolParam("Action: list, show, reset.") String action,
            @McpToolParam("Absolute path to the Git repo") String directory,
            @McpToolParam(value = "Git reference (default: HEAD).",
                    required = false) String reference,
            @McpToolParam(value = "Entry index (0-based; negatives "
                    + "count from end).", required = false) Integer index,
            @McpToolParam(value = "Max entries.",
                    required = false) Integer limit) { ... }
```

Live Demo: The Accidental Reset

Goal: Recover lost commits after a hard reset

AI-assisted workflow:

1. Ask AI: "I lost my work after a reset"
2. AI suggests checking reflog
3. AI helps interpret reflog output
4. AI recommends recovery command
5. You execute with confidence

```
# AI helps you understand:
$ git reflog
abc1234 HEAD@{0}: reset: moving to HEAD~5
def5678 HEAD@{1}: commit: Important feature work

# AI suggests:
$ git reset --hard def5678
```

Result: Work restored with AI guidance in seconds

Panic #2: "Which commit broke the build?"

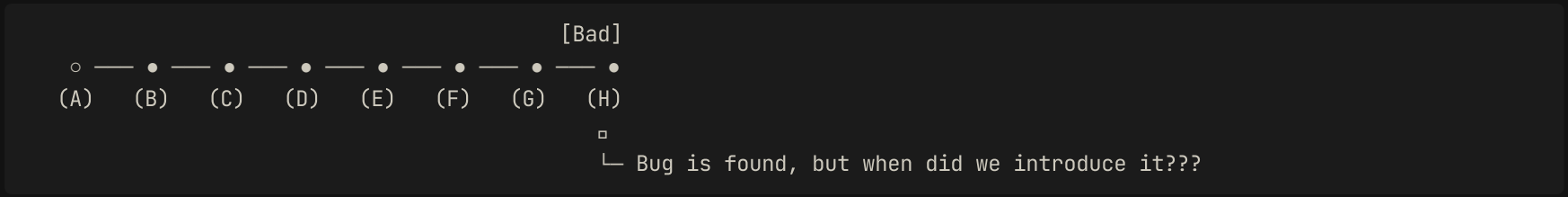
The login feature worked last Tuesday. It’s broken today. Somewhere in the last 40 commits someone introduced the bug, and the commit messages are too vague to guess.

Git's answer: `git bisect`

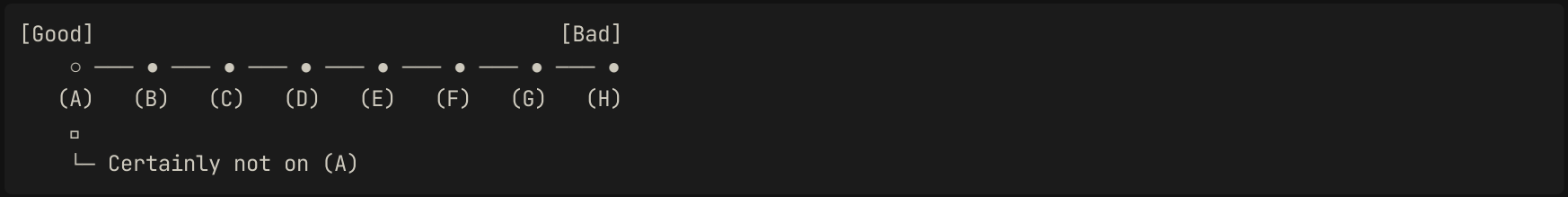
Binary search through history.

1. Mark a known-good commit (last Tuesday's tag).
2. Mark a known-bad commit (HEAD).
3. Git checks out the midpoint. You test it. You mark good or bad.
4. Repeat. In $\log_2(n)$ steps Git points at the culprit.

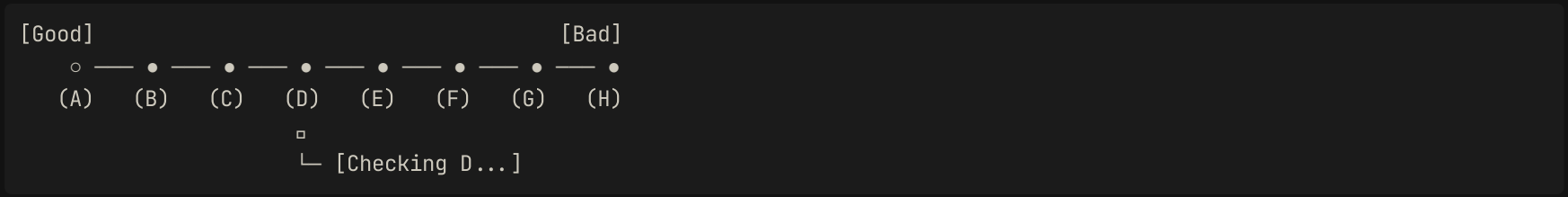
Visual walkthrough:



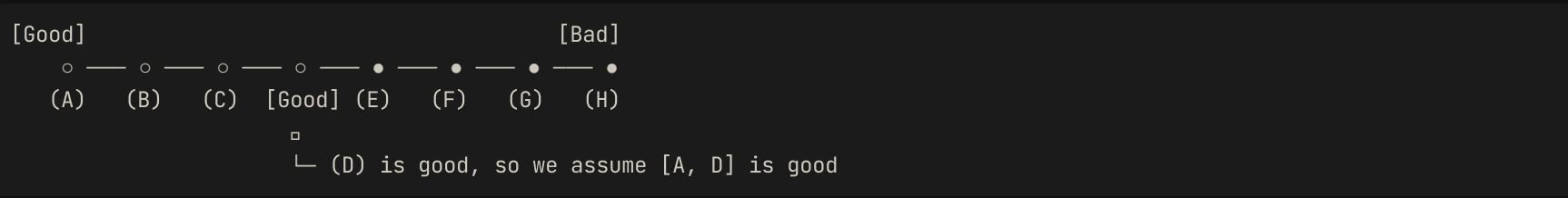
Visual walkthrough:



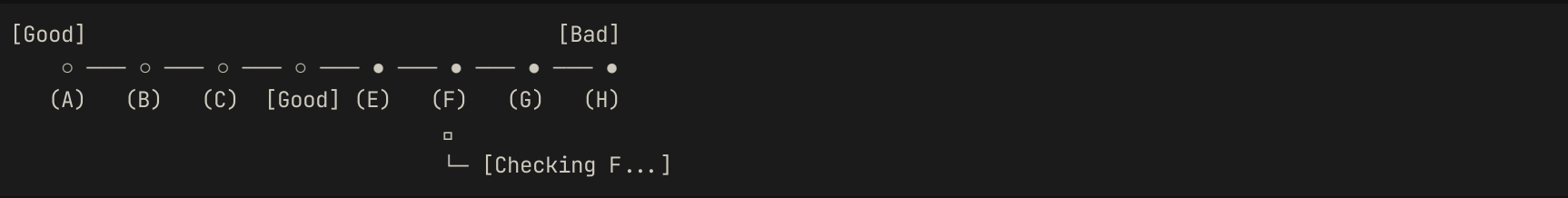
Visual walkthrough:



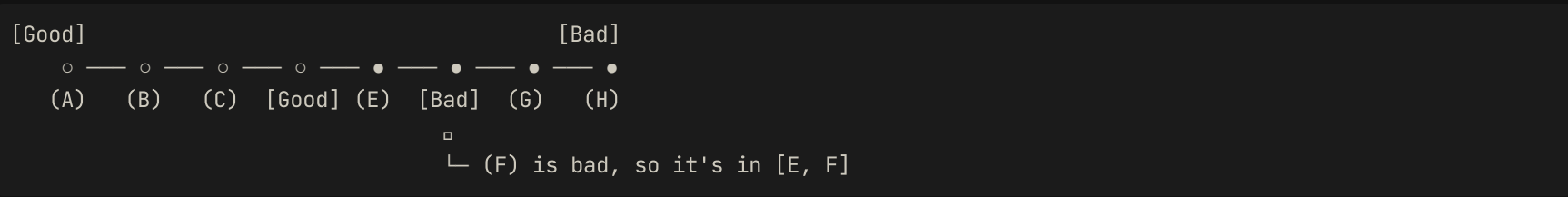
Visual walkthrough:



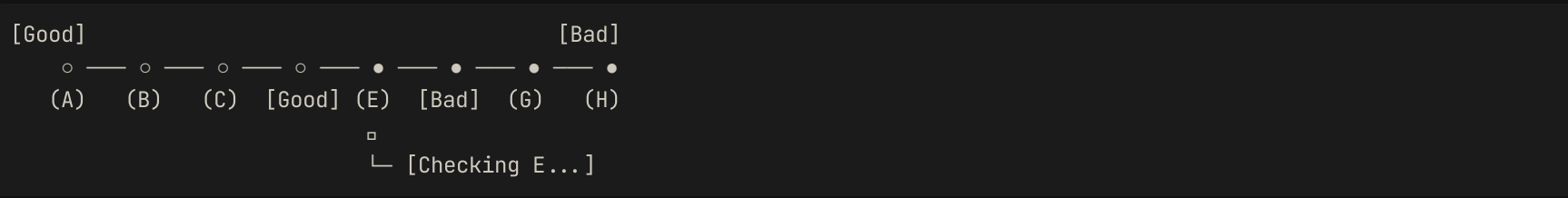
Visual walkthrough:



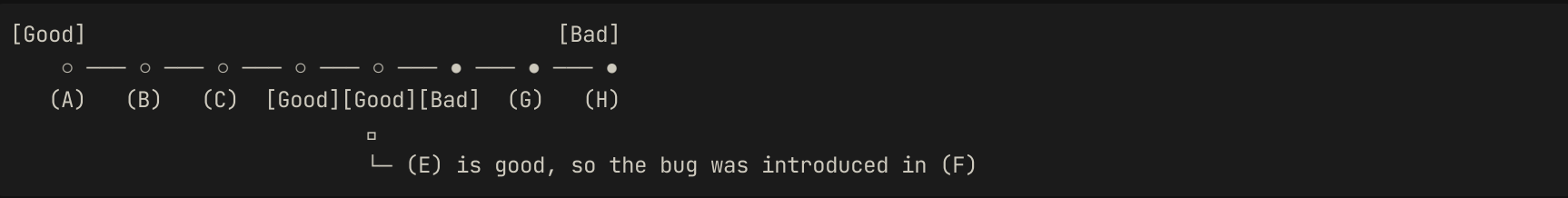
Visual walkthrough:



Visual walkthrough:



Visual walkthrough:



Commands

```
$ git bisect start
$ git bisect good abc123    # working login
$ git bisect bad def456     # broken login
# Git checks out midpoint automatically
$ ./run-login-test.sh       # test the feature
$ git bisect good           # test passed
# OR
$ git bisect bad            # test failed
# Repeat until found!
$ git bisect reset          # cleanup
```

Commands

```
$ git bisect start
$ git bisect good abc123    # working login
$ git bisect bad def456     # broken login
# Git checks out midpoint automatically
$ ./run-login-test.sh       # test the feature
$ git bisect good           # test passed
# OR
$ git bisect bad            # test failed
# Repeat until found!
$ git bisect reset          # cleanup
```

Find bugs in $O(\log_2(n))$ time instead of $O(n)$!

Java + MCP: `GitBisectService`

Bisect is stateful, so we will put markers in git to remember what we che

```
public record BisectOperationResult(
        String action,
        String message,
        BisectStatus status,
        CommitDetails currentCommit
) {
    public record CommitDetails(String hash, String timestamp, String message) {}
    public record BisectStatus(String good, String bad,
                               String current, int skippedCount) {}
}

@McpTool(name = "git_bisect",
    description = "Find the commit that introduced a bug.")
public BisectOperationResult gitBisect(
        @McpToolParam("Action: start|good|bad|skip|reset|status|view")
        String action,
        @McpToolParam("Absolute path") String directory,
        @McpToolParam(value = "Known-good", required = false) String good,
```

Java + MCP: `GitBisectService`

Bisect is stateful, so we will put markers in git to remember what we che

```
public record BisectOperationResult(
        String action,
        String message,
        BisectStatus status,
        CommitDetails currentCommit
) {
    public record CommitDetails(String hash, String timestamp, String message) {}
    public record BisectStatus(String good, String bad,
                               String current, int skippedCount) {}
}

@McpTool(name = "git_bisect",
    description = "Find the commit that introduced a bug.")
public BisectOperationResult gitBisect(
        @McpToolParam("Action: start|good|bad|skip|reset|status|view")
        String action,
        @McpToolParam("Absolute path") String directory,
        @McpToolParam(value = "Known-good", required = false) String good,
```

Java + MCP: `GitBisectService`

Bisect is stateful, so we will put markers in git to remember what we che

```
        BisectStatus status,
        CommitDetails currentCommit
) {
    public record CommitDetails(String hash, String timestamp, String message) {}
    public record BisectStatus(String good, String bad,
                               String current, int skippedCount) {}
}

@McpTool(name = "git_bisect",
    description = "Find the commit that introduced a bug.")
public BisectOperationResult gitBisect(
        @McpToolParam("Action: start|good|bad|skip|reset|status|view")
        String action,
        @McpToolParam("Absolute path") String directory,
        @McpToolParam(value = "Known-good", required = false) String good,
        @McpToolParam(value = "Known-bad",  required = false) String bad) {
    try (Git git = openGitRepository(directory)) {
        return switch (action) {
            case "start" -> startBisect(git, directory, good, bad);
```

Java + MCP: `GitBisectService`

Bisect is stateful, so we will put markers in git to remember what we che

```
8       public record BisectStatus(String good, String bad,
9                                  String current, int skippedCount) {}
10  }
11
12  @McpTool(name = "git_bisect",
13      description = "Find the commit that introduced a bug.")
14  public BisectOperationResult gitBisect(
15          @McpToolParam("Action: start|good|bad|skip|reset|status|view")
16          String action,
17          @McpToolParam("Absolute path") String directory,
18          @McpToolParam(value = "Known-good", required = false) String good,
19          @McpToolParam(value = "Known-bad",  required = false) String bad) {
20      try (Git git = openGitRepository(directory)) {
21          return switch (action) {
22              case "start"  -> startBisect(git, directory, good, bad);
23              case "good"   -> markGood(git, directory);
24              case "bad"    -> markBad(git, directory);
25              case "skip"   -> markSkip(git, directory);
26              case "reset"  -> resetBisect(git, directory);
```

Java + MCP: `GitBisectService`

Bisect is stateful, so we will put markers in git to remember what we che

```
16          String action,
17          @McpToolParam("Absolute path") String directory,
18          @McpToolParam(value = "Known-good", required = false) String good,
19          @McpToolParam(value = "Known-bad",  required = false) String bad) {
20      try (Git git = openGitRepository(directory)) {
21          return switch (action) {
22              case "start"  -> startBisect(git, directory, good, bad);
23              case "good"   -> markGood(git, directory);
24              case "bad"    -> markBad(git, directory);
25              case "skip"   -> markSkip(git, directory);
26              case "reset"  -> resetBisect(git, directory);
27              case "status" -> getStatus(directory);
28              case "view"   -> getCurrentCommit(git);
29              default -> throw new IllegalArgumentException(
30                      "Unknown action: " + action);
31          };
32      }
33  }
```

Live Demo: The Mysterious Bug

Goal: Find which commit broke the login feature

Live Demo: The Mysterious Bug

Goal: Find which commit broke the login feature

AI-assisted workflow:

1. Ask AI: "Find the commit that broke login"
2. **AI analyzes** commit history and suggests bisect plan
3. **AI guides** executes `git bisect start --good abc123 --bad HEAD` for you
4. **You run** the test at each step and tell AI results
5. AI identifies the offending commit

Live Demo: The Mysterious Bug

Goal: Find which commit broke the login feature

AI-assisted workflow:

1. Ask AI: "Find the commit that broke login"
2. **AI analyzes** commit history and suggests bisect plan
3. **AI guides** executes `git bisect start --good abc123 --bad HEAD` for you
4. **You run** the test at each step and tell AI results
5. AI identifies the offending commit

AI suggestions:

```
# Step 1: Start bisect
$ git bisect start
$ git bisect good abc123  # working commit
$ git bisect bad HEAD     # broken commit

# Step 2: Test & mark
$ ./test-login.sh        # run your test
$ git bisect good # or bad
```

Panic #3: "My pull request is a mess"

Panic #3: "My pull request is a mess"

- Ten WIP commits.

Panic #3: "My pull request is a mess"

- Ten WIP commits.
- Debug logs.

Panic #3: "My pull request is a mess"

- Ten WIP commits.
- Debug logs.
- Typo fixes.

Panic #3: "My pull request is a mess"

- Ten WIP commits.
- Debug logs.
- Typo fixes.
- Reviewer says "please clean this up before I merge."

Git’s answer: interactive rebase

Before: messy history

```
# Too many small commits!
feat: Add user service
fix: typo in service name
wip: validation logic
debug: log user creation
fix: null pointer exception
feat: add email validation
wip: test coverage
debug: fix test flakes
fix: email regex bug
feat: finish user service
```

After: clean history

```
# 3 logical commits
feat: Implement user service
feat: Add email validation
fix: Fix validation bugs
```

Before: messy history

```
# Too many small commits!
feat: Add user service
fix: typo in service name
wip: validation logic
debug: log user creation
fix: null pointer exception
feat: add email validation
wip: test coverage
debug: fix test flakes
fix: email regex bug
feat: finish user service
```

After: interactive rebase plan

```
pick   abc123 feat: Implement user service
squash def456 fix: typo in service name
squash ghi789 wip: validation logic
squash jkl012 debug: log user creation
squash mno345 fix: null pointer exception
pick   pqr678 feat: Add email validation
squash stu901 wip: test coverage
squash vwx234 debug: fix test flakes
squash yza567 fix: email regex bug
pick   bcd890 feat: finish user service
```

Java + MCP: `GitInteractiveRebaseService`

This is the complex one. A validated plan, an optional dry-run, an execute path. Let’s walk through it piece by piece.

First, the schema: a plain record

```
public record RebaseAction(
        String type,     // pick, reword, squash, fixup, drop, break
        String commit,   // required for everything except 'break'
        String message   // only for 'reword'
) {}
```

First, the schema: a plain record

```
public record RebaseAction(
        String type,     // pick, reword, squash, fixup, drop, break
        String commit,   // required for everything except 'break'
        String message   // only for 'reword'
) {}
```

First, the schema: a plain record

```
public record RebaseAction(
        String type,     // pick, reword, squash, fixup, drop, break
        String commit,   // required for everything except 'break'
        String message   // only for 'reword'
) {}
```

First, the schema: a plain record

```
public record RebaseAction(
        String type,      // pick, reword, squash, fixup, drop, break
        String commit,    // required for everything except 'break'
        String message    // only for 'reword'
) {}
```

First, the schema: a plain record

```
1   public record RebaseAction(
2           String type,      // pick, reword, squash, fixup, drop, break
3           String commit,    // required for everything except 'break'
4           String message    // only for 'reword'
5   ) {}
```

The LLM sees this exact schema. Jackson handles serialization, so the field names and types *are* the tool's contract.

`@McpTool` : annotate the method

```java
@McpTool(
    name = "git_interactive_rebase",
    description = "Perform interactive rebase: pick, reword, "
                + "edit, squash, fixup, drop, break."
)
public RebaseOperationResult gitInteractiveRebase(
        String directory,
        String upstream,
        List<RebaseAction> plan,
        Boolean dryRun) { ... }
```

`@McpTool`: annotate the method

```java
@McpTool(
    name = "git_interactive_rebase",
    description = "Perform interactive rebase: pick, reword, "
                + "edit, squash, fixup, drop, break."
)
public RebaseOperationResult gitInteractiveRebase(
        String directory,
        String upstream,
        List<RebaseAction> plan,
        Boolean dryRun) { ... }
```

`@McpTool`: annotate the method

```java
@McpTool(
    name = "git_interactive_rebase",
    description = "Perform interactive rebase: pick, reword, "
                + "edit, squash, fixup, drop, break."
)
public RebaseOperationResult gitInteractiveRebase(
        String directory,
        String upstream,
        List<RebaseAction> plan,
        Boolean dryRun) { ... }
```

`@McpTool` : annotate the method

```java
@McpTool(
    name = "git_interactive_rebase",
    description = "Perform interactive rebase: pick, reword, "
                + "edit, squash, fixup, drop, break."
)
public RebaseOperationResult gitInteractiveRebase(
        String directory,
        String upstream,
        List<RebaseAction> plan,
        Boolean dryRun) { ... }
```

But the LLM needs to know what each parameter means

Same method, now with an `@McpToolParam` description on every argument.

```
public RebaseOperationResult gitInteractiveRebase(
        String directory,
        String upstream,
        List<RebaseAction> plan,
        Boolean dryRun) { ... }
```

But the LLM needs to know what each parameter means

Same method, now with an `@McpToolParam` description on every argument.

```
public RebaseOperationResult gitInteractiveRebase(
        @McpToolParam(description = "Absolute path to a local Git repo")
        String directory,
        @McpToolParam(description = "Upstream ref to rebase onto "
                + "(default: HEAD~1). Use '--root' for a root rebase.",
                required = false)
        String upstream,
        @McpToolParam(description = "Rebase plan — list of actions.")
        List<RebaseAction> plan,
        @McpToolParam(description = "If true, validate & emit todo "
                + "file without executing.", required = false)
        Boolean dryRun) { ... }
```

But the LLM needs to know what each parameter means

Same method, now with an `@McpToolParam` description on every argument.

```
public RebaseOperationResult gitInteractiveRebase(
        @McpToolParam(description = "Absolute path to a local Git repo")
        String directory,
        @McpToolParam(description = "Upstream ref to rebase onto "
                + "(default: HEAD~1). Use '--root' for a root rebase.",
                required = false)
        String upstream,
        @McpToolParam(description = "Rebase plan — list of actions.")
        List<RebaseAction> plan,
        @McpToolParam(description = "If true, validate & emit todo "
                + "file without executing.", required = false)
        Boolean dryRun) { ... }
```

But the LLM needs to know what each parameter means

Same method, now with an `@McpToolParam` description on every argument.

```
public RebaseOperationResult gitInteractiveRebase(
        @McpToolParam(description = "Absolute path to a local Git repo")
        String directory,
        @McpToolParam(description = "Upstream ref to rebase onto "
                + "(default: HEAD~1). Use '--root' for a root rebase.",
                required = false)
        String upstream,
        @McpToolParam(description = "Rebase plan — list of actions.")
        List<RebaseAction> plan,
        @McpToolParam(description = "If true, validate & emit todo "
                + "file without executing.", required = false)
        Boolean dryRun) { ... }
```

These descriptions are part of the prompt sent to the model.

Write them like documentation for your future self *and* an LLM.

The body: validate, then dry-run or execute

```
1   if (upstream == null || upstream.isBlank()) upstream = "HEAD~1";
2   if (dryRun == null) dryRun = false;
3
4   validatePlan(directory, upstream, plan);
5
6   return dryRun
7       ? new RebaseOperationResult("DRY_RUN",
8               "Todo file generated.", null,
9               generateTodoFile(directory, upstream, plan))
10      : executeRebase(directory, upstream, plan);
```

The body: validate, then dry-run or execute

```
1   if (upstream == null || upstream.isBlank()) upstream = "HEAD~1";
2   if (dryRun == null) dryRun = false;
3
4   validatePlan(directory, upstream, plan);
5
6   return dryRun
7       ? new RebaseOperationResult("DRY_RUN",
8               "Todo file generated.", null,
9               generateTodoFile(directory, upstream, plan))
10      : executeRebase(directory, upstream, plan);
```

The body: validate, then dry-run or execute

```
1   if (upstream == null || upstream.isBlank()) upstream = "HEAD~1";
2   if (dryRun == null) dryRun = false;
3
4   validatePlan(directory, upstream, plan);
5
6   return dryRun
7       ? new RebaseOperationResult("DRY_RUN",
8               "Todo file generated.", null,
9               generateTodoFile(directory, upstream, plan))
10      : executeRebase(directory, upstream, plan);
```

The body: validate, then dry-run or execute

```
1   if (upstream == null || upstream.isBlank()) upstream = "HEAD~1";
2   if (dryRun == null) dryRun = false;
3
4   validatePlan(directory, upstream, plan);
5
6   return dryRun
7       ? new RebaseOperationResult("DRY_RUN",
8               "Todo file generated.", null,
9               generateTodoFile(directory, upstream, plan))
10      : executeRebase(directory, upstream, plan);
```

The body: validate, then dry-run or execute

```
1   if (upstream == null || upstream.isBlank()) upstream = "HEAD~1";
2   if (dryRun == null) dryRun = false;
3
4   validatePlan(directory, upstream, plan);
5
6   return dryRun
7       ? new RebaseOperationResult("DRY_RUN",
8               "Todo file generated.", null,
9               generateTodoFile(directory, upstream, plan))
10      : executeRebase(directory, upstream, plan);
```

The body: validate, then dry-run or execute

```
1   if (upstream == null || upstream.isBlank()) upstream = "HEAD~1";
2   if (dryRun == null) dryRun = false;
3
4   validatePlan(directory, upstream, plan);
5
6   return dryRun
7       ? new RebaseOperationResult("DRY_RUN",
8               "Todo file generated.", null,
9               generateTodoFile(directory, upstream, plan))
10      : executeRebase(directory, upstream, plan);
```

- Normalise nullable optionals first. LLMs love to omit them.

The body: validate, then dry-run or execute

```
1   if (upstream == null || upstream.isBlank()) upstream = "HEAD~1";
2   if (dryRun == null) dryRun = false;
3
4   validatePlan(directory, upstream, plan);
5
6   return dryRun
7       ? new RebaseOperationResult("DRY_RUN",
8               "Todo file generated.", null,
9               generateTodoFile(directory, upstream, plan))
10      : executeRebase(directory, upstream, plan);
```

- Normalise nullable optionals first. LLMs love to omit them.
- Validate before you touch the repo. Throw `IllegalArgumentException` with a human message and Spring AI will surface it back to the LLM.

The body: validate, then dry-run or execute

```
1   if (upstream == null || upstream.isBlank()) upstream = "HEAD~1";
2   if (dryRun == null) dryRun = false;
3
4   validatePlan(directory, upstream, plan);
5
6   return dryRun
7       ? new RebaseOperationResult("DRY_RUN",
8               "Todo file generated.", null,
9               generateTodoFile(directory, upstream, plan))
10      : executeRebase(directory, upstream, plan);
```

- Normalise nullable optionals first. LLMs love to omit them.
- Validate before you touch the repo. Throw `IllegalArgumentException` with a human message and Spring AI will surface it back to the LLM.
- Honour `dryRun`. It returns the generated todo file without touching the repo, and it's the safety net that makes agents usable.

The body: validate, then dry-run or execute

```
1   if (upstream == null || upstream.isBlank()) upstream = "HEAD~1";
2   if (dryRun == null) dryRun = false;
3
4   validatePlan(directory, upstream, plan);
5
6   return dryRun
7       ? new RebaseOperationResult("DRY_RUN",
8               "Todo file generated.", null,
9               generateTodoFile(directory, upstream, plan))
10      : executeRebase(directory, upstream, plan);
```

- Normalise nullable optionals first. LLMs love to omit them.
- Validate before you touch the repo. Throw `IllegalArgumentException` with a human message and Spring AI will surface it back to the LLM.
- Honour `dryRun`. It returns the generated todo file without touching the repo, and it's the safety net that makes agents usable.
- Only run `executeRebase` once validation passed and `dryRun` is explicitly false.

The return type: structured, not text

```java
public record RebaseOperationResult(
        String status,    // SUCCESS, DRY_RUN, CONFLICTS
        String message,
        String newHead,
        String todoFile
) {}

// Returned as JSON automatically. No prompt parsing.
// The LLM sees typed fields and can branch on `status`
// without hoping a regex holds.
```

The return type: structured, not text

```java
public record RebaseOperationResult(
        String status,    // SUCCESS, DRY_RUN, CONFLICTS
        String message,
        String newHead,
        String todoFile
) {}

// Returned as JSON automatically. No prompt parsing.
// The LLM sees typed fields and can branch on `status`
// without hoping a regex holds.
```

The return type: structured, not text

```
public record RebaseOperationResult(
        String status,    // SUCCESS, DRY_RUN, CONFLICTS
        String message,
        String newHead,
        String todoFile
) {}

// Returned as JSON automatically. No prompt parsing.
// The LLM sees typed fields and can branch on `status`
// without hoping a regex holds.
```

The return type: structured, not text

```
public record RebaseOperationResult(
        String status,    // SUCCESS, DRY_RUN, CONFLICTS
        String message,
        String newHead,
        String todoFile
) {}

// Returned as JSON automatically. No prompt parsing.
// The LLM sees typed fields and can branch on `status`
// without hoping a regex holds.
```

The return type: structured, not text

```
public record RebaseOperationResult(
        String status,    // SUCCESS, DRY_RUN, CONFLICTS
        String message,
        String newHead,
        String todoFile
) {}

// Returned as JSON automatically. No prompt parsing.
// The LLM sees typed fields and can branch on `status`
// without hoping a regex holds.
```

Records give the LLM a stable schema instead of free-form prose.

Typed output is what makes tool chaining reliable.

Live Demo: The Messy PR

Goal: Clean up 10 messy commits into 3 logical commits

AI-assisted workflow:

1. Ask AI: "Clean up my PR history"
2. AI analyzes commits with `git_log`
3. AI generates rebase plan (squash, reword)
4. AI shows dry-run plan for your review
5. You execute the validated plan

Before:

```
Add user service
fix typo
wip
debug logs
finish user service
```

AI calls `git_interactive_rebase` with:

```
[
  { "type": "pick",   "commit": "abc123" },
  { "type": "squash", "commit": "def456" },
  { "type": "squash", "commit": "ghi789" },
  { "type": "reword", "commit": "jkl012",
    "message": "feat: Implement user service" },
  { "type": "pick",   "commit": "mno345" },
  { "type": "squash", "commit": "pqr678" }
]
```

Matches the real `RebaseAction` record. Call it with `dryRun: true` first, then execute.

Live Demo: The Messy PR

Goal: Clean up 10 messy commits into 3 logical commits

AI-assisted workflow:

1. Ask AI: "Clean up my PR history"
2. AI analyzes commits with `git_log`
3. AI generates rebase plan (squash, reword)
4. AI shows dry-run plan for your review
5. You execute the validated plan

Before:

```
Add user service
fix typo
wip
debug logs
finish user service
```

AI calls `git_interactive_rebase` with:

```
[
  { "type": "pick",   "commit": "abc123" },
  { "type": "squash", "commit": "def456" },
  { "type": "squash", "commit": "ghi789" },
  { "type": "reword", "commit": "jkl012",
    "message": "feat: Implement user service" },
  { "type": "pick",   "commit": "mno345" },
  { "type": "squash", "commit": "pqr678" }
]
```

Matches the real `RebaseAction` record. Call it with `dryRun: true` first, then execute.

Live Demo: The Messy PR

Goal: Clean up 10 messy commits into 3 logical commits

AI-assisted workflow:

1. Ask AI: "Clean up my PR history"
2. AI analyzes commits with `git_log`
3. AI generates rebase plan (squash, reword)
4. AI shows dry-run plan for your review
5. You execute the validated plan

Before:

```
Add user service
fix typo
wip
debug logs
finish user service
```

AI calls `git_interactive_rebase` with:

```
[
  { "type": "pick",   "commit": "abc123" },
  { "type": "squash", "commit": "def456" },
  { "type": "squash", "commit": "ghi789" },
  { "type": "reword", "commit": "jkl012",
    "message": "feat: Implement user service" },
  { "type": "pick",   "commit": "mno345" },
  { "type": "squash", "commit": "pqr678" }
]
```

Matches the real `RebaseAction` record. Call it with `dryRun: true` first, then execute.

Live Demo: The Messy PR

Goal: Clean up 10 messy commits into 3 logical commits

AI-assisted workflow:

1. Ask AI: "Clean up my PR history"
2. AI analyzes commits with `git_log`
3. AI generates rebase plan (squash, reword)
4. AI shows dry-run plan for your review
5. You execute the validated plan

Before:

```
Add user service
fix typo
wip
debug logs
finish user service
```

AI calls `git_interactive_rebase` with:

```
[
  { "type": "pick",   "commit": "abc123" },
  { "type": "squash", "commit": "def456" },
  { "type": "squash", "commit": "ghi789" },
  { "type": "reword", "commit": "jkl012",
    "message": "feat: Implement user service" },
  { "type": "pick",   "commit": "mno345" },
  { "type": "squash", "commit": "pqr678" }
]
```

Matches the real `RebaseAction` record. Call it with `dryRun: true` first, then execute.

Beyond the tools

Patterns worth stealing

Bootstrap: `main` plus AOT hints

```java
@SpringBootApplication
@ImportRuntimeHints(GithubMcpApplication.JGitRuntimeHints.class)
public class GithubMcpApplication {

    public static void main(String[] args) {
        SpringApplication.run(GithubMcpApplication.class, args);
    }

    // Native image: JGit uses reflection and ServiceLoader heavily.
    public static class JGitRuntimeHints implements RuntimeHintsRegistrar {
        @Override
        public void registerHints(RuntimeHints hints, ClassLoader cl) {
            Stream.of(
                    McpSchema.Tool.class,
                    GitBisectService.class,
                    GitInteractiveRebaseService.class,
                    GitInteractiveRebaseService.RebaseAction.class
                    /* ... */)
                .forEach(t -> hints.reflection().registerType(t,
                    MemberCategory.INVOKE_DECLARED_CONSTRUCTORS,
                    MemberCategory.INVOKE_DECLARED_METHODS));
```

Bootstrap: `main` plus AOT hints

```java
@SpringBootApplication
@ImportRuntimeHints(GithubMcpApplication.JGitRuntimeHints.class)
public class GithubMcpApplication {

    public static void main(String[] args) {
        SpringApplication.run(GithubMcpApplication.class, args);
    }

    // Native image: JGit uses reflection and ServiceLoader heavily.
    public static class JGitRuntimeHints implements RuntimeHintsRegistrar {
        @Override
        public void registerHints(RuntimeHints hints, ClassLoader cl) {
            Stream.of(
                    McpSchema.Tool.class,
                    GitBisectService.class,
                    GitInteractiveRebaseService.class,
                    GitInteractiveRebaseService.RebaseAction.class
                    /* ... */)
                .forEach(t -> hints.reflection().registerType(t,
                    MemberCategory.INVOKE_DECLARED_CONSTRUCTORS,
                    MemberCategory.INVOKE_DECLARED_METHODS));
```

Bootstrap: `main` plus AOT hints

```java
@SpringBootApplication
@ImportRuntimeHints(GithubMcpApplication.JGitRuntimeHints.class)
public class GithubMcpApplication {

    public static void main(String[] args) {
        SpringApplication.run(GithubMcpApplication.class, args);
    }

    // Native image: JGit uses reflection and ServiceLoader heavily.
    public static class JGitRuntimeHints implements RuntimeHintsRegistrar {
        @Override
        public void registerHints(RuntimeHints hints, ClassLoader cl) {
            Stream.of(
                    McpSchema.Tool.class,
                    GitBisectService.class,
                    GitInteractiveRebaseService.class,
                    GitInteractiveRebaseService.RebaseAction.class
                    /* ... */)
                .forEach(t -> hints.reflection().registerType(t,
                    MemberCategory.INVOKE_DECLARED_CONSTRUCTORS,
                    MemberCategory.INVOKE_DECLARED_METHODS));
```

Bootstrap: `main` plus AOT hints

```java
@SpringBootApplication
@ImportRuntimeHints(GithubMcpApplication.JGitRuntimeHints.class)
public class GithubMcpApplication {

    public static void main(String[] args) {
        SpringApplication.run(GithubMcpApplication.class, args);
    }

    // Native image: JGit uses reflection and ServiceLoader heavily.
    public static class JGitRuntimeHints implements RuntimeHintsRegistrar {
        @Override
        public void registerHints(RuntimeHints hints, ClassLoader cl) {
            Stream.of(
                    McpSchema.Tool.class,
                    GitBisectService.class,
                    GitInteractiveRebaseService.class,
                    GitInteractiveRebaseService.RebaseAction.class
                    /* ... */)
                .forEach(t -> hints.reflection().registerType(t,
                    MemberCategory.INVOKE_DECLARED_CONSTRUCTORS,
                    MemberCategory.INVOKE_DECLARED_METHODS));
```

Bootstrap: `main` plus AOT hints

```
2   @ImportRuntimeHints(GithubMcpApplication.JGitRuntimeHints.class)
3   public class GithubMcpApplication {
4
5       public static void main(String[] args) {
6           SpringApplication.run(GithubMcpApplication.class, args);
7       }
8
9       // Native image: JGit uses reflection and ServiceLoader heavily.
10      public static class JGitRuntimeHints implements RuntimeHintsRegistrar {
11          @Override
12          public void registerHints(RuntimeHints hints, ClassLoader cl) {
13              Stream.of(
14                      McpSchema.Tool.class,
15                      GitBisectService.class,
16                      GitInteractiveRebaseService.class,
17                      GitInteractiveRebaseService.RebaseAction.class
18                      /* ... */)
19                  .forEach(t → hints.reflection().registerType(t,
20                      MemberCategory.INVOKE_DECLARED_CONSTRUCTORS,
21                      MemberCategory.INVOKE_DECLARED_METHODS));
22
```

Bootstrap: `main` plus AOT hints

```
5       public static void main(String[] args) {
6           SpringApplication.run(GithubMcpApplication.class, args);
7       }
8
9       // Native image: JGit uses reflection and ServiceLoader heavily.
10      public static class JGitRuntimeHints implements RuntimeHintsRegistrar {
11          @Override
12          public void registerHints(RuntimeHints hints, ClassLoader cl) {
13              Stream.of(
14                      McpSchema.Tool.class,
15                      GitBisectService.class,
16                      GitInteractiveRebaseService.class,
17                      GitInteractiveRebaseService.RebaseAction.class
18                      /* ... */)
19                  .forEach(t -> hints.reflection().registerType(t,
20                      MemberCategory.INVOKE_DECLARED_CONSTRUCTORS,
21                      MemberCategory.INVOKE_DECLARED_METHODS));
22
23              hints.resources().registerPattern(
24                      "META-INF/services/org.eclipse.jgit.*");
25          }
```

Bootstrap: `main` plus AOT hints

```
    }

    // Native image: JGit uses reflection and ServiceLoader heavily.
    public static class JGitRuntimeHints implements RuntimeHintsRegistrar {
        @Override
        public void registerHints(RuntimeHints hints, ClassLoader cl) {
            Stream.of(
                    McpSchema.Tool.class,
                    GitBisectService.class,
                    GitInteractiveRebaseService.class,
                    GitInteractiveRebaseService.RebaseAction.class
                    /* ... */)
                .forEach(t -> hints.reflection().registerType(t,
                    MemberCategory.INVOKE_DECLARED_CONSTRUCTORS,
                    MemberCategory.INVOKE_DECLARED_METHODS));

            hints.resources().registerPattern(
                    "META-INF/services/org.eclipse.jgit.*");
        }
    }
}
```

Bootstrap: `main` plus AOT hints

```
    }

    // Native image: JGit uses reflection and ServiceLoader heavily.
    public static class JGitRuntimeHints implements RuntimeHintsRegistrar {
        @Override
        public void registerHints(RuntimeHints hints, ClassLoader cl) {
            Stream.of(
                    McpSchema.Tool.class,
                    GitBisectService.class,
                    GitInteractiveRebaseService.class,
                    GitInteractiveRebaseService.RebaseAction.class
                    /* ... */)
                .forEach(t -> hints.reflection().registerType(t,
                    MemberCategory.INVOKE_DECLARED_CONSTRUCTORS,
                    MemberCategory.INVOKE_DECLARED_METHODS));

            hints.resources().registerPattern(
                    "META-INF/services/org.eclipse.jgit.*");
        }
    }
}
```

Tool-calling vs Chat-only AI

Tool-calling vs Chat-only AI

Chat-only AI:

- Can only suggest commands
- No validation, no safety checks
- Manual copy-paste with risk

Tool-calling vs Chat-only AI

Chat-only AI:

- Can only suggest commands
- No validation, no safety checks
- Manual copy-paste with risk

Tool-calling AI:

- Validates operations before suggestion
- Handles stateful workflows (bisect, rebase)
- Provides safety (dry-run, confirmations)

Tool-calling vs Chat-only AI

Chat-only AI:

- Can only suggest commands
- No validation, no safety checks
- Manual copy-paste with risk

Tool-calling AI:

- Validates operations before suggestion
- Handles stateful workflows (bisect, rebase)
- Provides safety (dry-run, confirmations)

What changes in practice:

- AI reads the tool schema, plans a sequence, and executes it
- You stop copy-pasting commands from ChatGPT and hoping for the best

Security & Guardrails

What AI guides you away from:

- `git push --force` (destructive)
- Deleting branches without backup
- Risky operations without confirmation

Implementation:

- Tool-level validation in Spring AI
- Dry-run modes for dangerous operations
- Scope restrictions (directory, repository)

Philosophy: AI as expert guide, not autonomous agent

Observability

Monitor AI guidance patterns:

- Spring Boot Actuator endpoints
- "Most-suggested tools" metrics
- Error reduction statistics

What you learn:

- Which tools get called most (bisect wins, every time)
- Where the LLM retries or fails
- What to fix in your tool descriptions

Conclusion & Q&A

Key Takeaways

Key Takeaways

1. **Git already has the tools — you just need to remember they exist**
   - Reflog: your safety net
   - Bisect: binary search for bugs
   - Interactive rebase: clean up before you merge

Key Takeaways

1. **Git already has the tools — you just need to remember they exist**
   - Reflog: your safety net
   - Bisect: binary search for bugs
   - Interactive rebase: clean up before you merge

2. **AI handles the bookkeeping, you make the decisions**
   - It reads reflog output so you don’t have to parse it under stress
   - It drives the bisect loop while you just run the test
   - It proposes a rebase plan; you approve before anything runs

Key Takeaways

1. **Git already has the tools — you just need to remember they exist**
   - Reflog: your safety net
   - Bisect: binary search for bugs
   - Interactive rebase: clean up before you merge

2. **AI handles the bookkeeping, you make the decisions**
   - It reads reflog output so you don't have to parse it under stress
   - It drives the bisect loop while you just run the test
   - It proposes a rebase plan; you approve before anything runs

3. **One annotation, one record, and you have an MCP tool**
   - `@McpTool` + `@McpToolParam` is the whole API surface
   - Java records go in and out as JSON — no DTOs, no mapping
   - ~1200 lines for three non-trivial Git tools

Thank You!

Questions?

Resources:

- Spring Git MCP repository: github.com/your-repo/spring-git-mcp
- Model Context Protocol: modelcontextprotocol.io
- Spring AI documentation: docs.spring.io/spring-ai

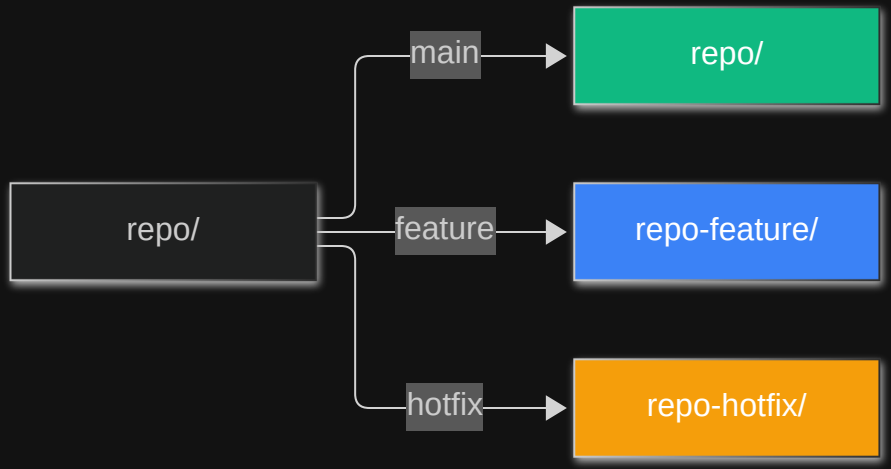
Contact:
me@asm0dey.site | @asm0di0 | https://asm0dey.site

Bonus: More Git Superpowers

Ideas for your next MCP tools

`git worktree` — parallel branches without stashing

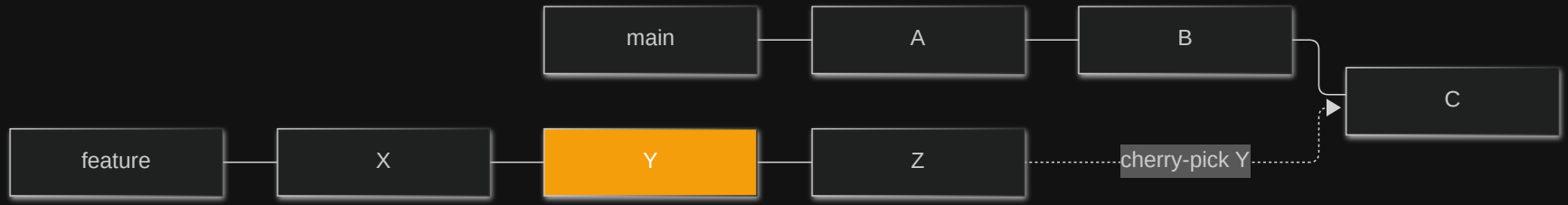
Work on two branches at the same time in separate directories, no `stash` juggling.



Panic it solves: "I need to review a PR but I have uncommitted work"

`git cherry-pick` — surgical commit transplant

Copy a single commit from one branch to another without merging the whole branch.



Panic it solves: "I need *that one fix* from the feature branch in production — now"

`git stash` — the pocket dimension

Temporarily shelve uncommitted changes and restore them later. Stashes stack.

`git blame` — line-level archaeology

Find who changed each line, when, and in which commit. The ultimate "why is this here?" tool.

```
$ git blame src/Auth.java
^abc123 (Alice  2025-03-01)  if (token == null) {
def4567 (Bob    2025-06-15)      throw new AuthException("expired");
^abc123 (Alice  2025-03-01)  }
ghi8901 (Carol  2025-09-22)  // TODO: refresh token logic
```

Panic it solves: "Who wrote this and what were they thinking?"

`git rerere` — remember conflict resolutions

"**Re**use **re**corded **re**solution." Git memorises how you resolved a conflict and auto-applies it next time the same conflict appears.



Panic it solves: "I keep resolving the same merge conflict over and over during long-lived rebases"

`git filter-repo` — full history surgery

Rewrite the entire repository history: remove large files, rename paths, strip secrets. The modern replacement for `filter-branch`.

```
# Remove accidentally committed credentials from ALL history
git filter-repo --path secrets.env --invert-paths

# Rename a directory across all commits
git filter-repo --path-rename old-name/:new-name/
```

Panic it solves: "Someone committed a 500MB binary / API key to the repo three months ago"